

# **LAPORAN PRAKTIKUM**

**Aplikasi Berbasis Platform**

**MODUL 10 & 11**

**LARAVEL & AJAX**



**Disusun oleh:**

**Muhammad Rafiful Hana**

**2311102227**

**S1 IF-11-REG04**

**Dosen Pengampu:**

**Cahyo Prihantoro, S.Kom., M.Eng**

**PROGRAM STUDI S1 INFORMATIKA**

**FAKULTAS INFORMATIKA**

**TELKOM UNIVERSITY PURWOKERTO**

**2026**

## DASAR TEORI

### A. AJAX

AJAX adalah teknik dalam pengembangan web yang memungkinkan pertukaran data antara browser dan server dilakukan secara asynchronous tanpa perlu memuat ulang seluruh halaman. AJAX biasanya menggunakan JavaScript untuk mengirim request ke server dan menerima respons dalam format seperti JSON atau XML, sehingga meningkatkan pengalaman pengguna menjadi lebih cepat dan responsif. Dengan AJAX, bagian tertentu dari halaman web dapat diperbarui secara dinamis tanpa refresh penuh, yang sangat berguna pada aplikasi modern seperti sistem chat, pencarian real-time, dan dashboard interaktif.

### B. Laravel

*Laravel* adalah framework pengembangan aplikasi web berbasis PHP yang bersifat open-source dan menggunakan pola arsitektur *MVC (Model-View-Controller)*. Laravel dirancang untuk mempermudah proses pengembangan aplikasi dengan menyediakan fitur bawaan seperti routing, ORM (Eloquent), middleware, authentication, dan templating engine Blade. Dengan struktur yang rapi dan sintaks yang ekspresif, Laravel membantu pengembang membangun aplikasi web yang kompleks menjadi lebih terorganisir, aman, dan mudah dipelihara. Framework ini juga memiliki ekosistem yang kuat seperti *Laravel Sanctum*, *Laravel Jetstream*, dan *Laravel Nova* yang mendukung pengembangan aplikasi modern secara lebih cepat dan efisien.

## UNGUIDED

### 1. PENDAHULUAN

Halo! Jadi gini, untuk tugas praktikum kali ini saya membuat aplikasi sederhana berbasis Laravel yang dikombinasikan dengan AJAX. Tema yang saya angkat adalah manajemen data mahasiswa dengan tampilan yang agak beda dari biasanya — saya pakai gaya brutalist (kasar, tegas, banyak kotak dan garis) biar kelihatan keren dan beda. Ide ini murni dari saya sendiri karena saya suka desain yang simpel tapi berkarakter.

### 2. KONSEP & ALUR PROGRAM

Saya membuat aplikasi CRUD (Create, Read, Update, Delete) untuk data mahasiswa. Bedanya, data ini disimpan di file JSON lokal, bukan di database MySQL seperti biasanya. Kenapa? Karena biar lebih simpel dan gak ribet setting database, plus ini sesuai dengan permintaan tugas untuk menggunakan JSON sebagai sumber data.

1. **Data disimpan di file JSON** - Semua data mahasiswa disimpan di `storage/app/mahasiswa.json`. Awalnya file ini kosong, tapi kalau gak ada, sistem akan otomatis bikin data dummy biar langsung bisa dipake.
2. **AJAX untuk semua interaksi** - Gak ada reload halaman sama sekali. Semua proses tambah, edit, hapus, dan tampil data pake AJAX. Jadi pengalaman ngaksesnya lebih smooth dan cepat.
3. **Validasi tetap jalan** - Walaupun pake JSON, validasi input tetep ada. Misalnya NIM harus 10 digit, email harus valid, hobi minimal pilih 1, dll. Ini penting biar data yang masuk bersih dan sesuai.

#### Struktur Data Mahasiswa:

##### Setiap mahasiswa punya data:

- Nama lengkap
- NIM (10 digit angka)
- Email
- Jenis kelamin (Pria/Wanita)
- Hobi (bisa pilih lebih dari 1, disimpan sebagai array)
- Pendidikan (D3/S1/S2/S3)

```
[
  {
    "id": 1,
    "nama": "Muhammad Rafiful Hana",
    "nim": 2311102227,
    "email": "rafifulhan@gmail.com",
    "jenis_kelamin": "Pria",
    "hobi": [
      "Coding",
      "Membaca",
      "Musik",
      "Traveling",
      "Fotografi"
    ],
    "pendidikan": "S1"
  },
]
```

Kenapa pake hobi dalam bentuk array? Karena dalam satu mahasiswa bisa punya banyak hobi, jadi lebih fleksibel daripada cuma satu kolom.

### 3. PROGRAM & PENJELASAN

#### a. Routing (routes/web.php)

Source Code:

```
<?php
use Illuminate\Support\Facades\Route;
use App\Http\Controllers\MahasiswaController;
Route::get('/', [MahasiswaController::class, 'index']);
Route::get('/mahasiswa', [MahasiswaController::class, 'index']);
Route::get('/mahasiswa/data', [MahasiswaController::class, 'getData']);
Route::post('/mahasiswa', [MahasiswaController::class, 'store']);
Route::put('/mahasiswa/{id}', [MahasiswaController::class, 'update']);
Route::delete('/mahasiswa/{id}', [MahasiswaController::class, 'destroy']);
```

#### Penjelasan:

Di sini saya mendefinisikan semua endpoint yang dibutuhkan. Ada 5 route utama: untuk menampilkan halaman, ambil data, tambah, edit, dan hapus. Method HTTP-nya sudah disesuaikan dengan fungsinya masing-masing (GET, POST, PUT, DELETE). Ini penting karena AJAX akan memanggil endpoint-endpoint ini.

#### b. Controller (MahasiswaController.php) - Read Write

Source Code:

```
private function getJsonPath()
{
    return storage_path('app/mahasiswa.json');
}
private function readData()
{
    $path = $this->getJsonPath();
    if (!File::exists($path)) {
        $default = [
            [
                'id' => 1,
                'nama' => 'Muhammad Rafiful Hana',
                'nim' => 2311102227,
                'email' => 'rafiful@example.com',
                'jenis_kelamin' => 'Pria',
                'hobi' => ['Coding', 'Gaming'],
                'pendidikan' => 'S1'
            ],
            [
                'id' => 2,
                'nama' => 'Ahmad Fauzi',
                'nim' => 2311102228,
                'email' => 'ahmad2@example.com',
                'jenis_kelamin' => 'Pria',
                'hobi' => ['Membaca', 'Menulis'],
                'pendidikan' => 'S1'
            ]
        ];
        File::put($path, json_encode($default, JSON_PRETTY_PRINT));
        return $default;
    }
    return json_decode(File::get($path), true) ?? [];
}
```

#### Penjelasan:

Ini adalah jantung dari aplikasi. `readData()` berfungsi untuk membaca file JSON dan mengubahnya jadi array PHP. Kalau file gak ada, saya bikin data default biar langsung bisa dipake. `writeData()` kebalikannya, mengubah array jadi JSON dan simpan ke file. Dengan ini, kita bisa "memalsukan" database pake file biasa.

**c. Controller (MahasiswaController.php) - Store / Create**  
**Source Code:**

```
public function store(Request $request)
{
    $validator = Validator::make($request->all(), [
        'nama' => 'required|string|max:100',
        'nim' => 'required|numeric|digits:10',
        'email' => 'required|email|max:100',
        'jenis_kelamin' => 'required|in:Pria,Wanita',
        'hobi' => 'required|array|min:1',
        'hobi.*' => 'string|max:50',
        'pendidikan' => 'required|in:S1,S2,S3,D3'
    ]);

    if ($validator->fails()) {
        return response()->json(['errors' => $validator->errors()], 422);
    }

    $data = $this->readData();
    $newId = count($data) > 0 ? max(array_column($data, 'id')) + 1 : 1;

    $newData = [
        'id' => $newId,
        'nama' => $request->nama,
        'nim' => (int)$request->nim,
        'email' => $request->email,
        'jenis_kelamin' => $request->jenis_kelamin,
        'hobi' => $request->hobi,
        'pendidikan' => $request->pendidikan
    ];

    $data[] = $newData;
    $this->writeData($data);

    return response()->json(['message' => 'Data berhasil ditambahkan', 'data' =>
    $newData], 201);
}
```

**Penjelasan:**

Proses tambah data dimulai dengan validasi - saya cek semua input harus sesuai format. Kalau ada yang salah, kirim balik error 422. Kalau lolos validasi, saya baca data JSON yang ada, cari ID terbaru + 1 buat ID baru, lalu tambahkan data baru ke array dan simpan lagi ke JSON. Response dikirim dalam format JSON biar bisa diproses AJAX.

**d. AJAX (Load Data)**  
**Source Code:**

```
function loadData() {
    $.ajax({
        url: baseUrl + '/data',
        method: 'GET',
        success: function(response) {
            renderTable(response);
        },
        error: function() {
            showToast('Gagal memuat data', 'error');
        }
    });
}
```

```

function renderTable(data) {
  if (!data || data.length === 0) {
    $('#tableContainer').html('<div class="empty-state"><i class="fas fa-database"></i>Belum ada data mahasiswa</div>');
    return;
  }

  let html = `<div style="overflow-x: auto;">
    <table class="table-brutal">
      <thead>
        <tr>
          <th>No</th>
          <th>Nama</th>
          <th>NIM</th>
          <th>Email</th>
          <th>JK</th>
          <th>Hobi</th>
          <th>Pend</th>
          <th style="text-align: center;">Aksi</th>
        </tr>
      </thead>
      <tbody>`;

  data.forEach((item, index) => {
    const genderClass = item.jenis_kelamin === 'Pria' ? 'pria' : 'wanita';
    const hobiHtml = item.hobi.map(h => `<span class="hobi-tag">${h}</span>`).join(' ');

    html += `<tr>
      <td>${index + 1}</td>
      <td><strong>${item.nama}</strong></td>
      <td>${item.nim}</td>
      <td>${item.email}</td>
      <td><span class="gender-badge ${genderClass}">${item.jenis_kelamin}</span></td>
      <td style="min-width: 120px;">${hobiHtml}</td>
      <td><strong>${item.pendidikan}</strong></td>
      <td style="text-align: center;">
        <div class="action-group">
          <button class="btn-brutal btn-sm btn-edit" data-id="${item.id}">
            <i class="fas fa-edit"></i>
          </button>
          <button class="btn-brutal btn-sm btn-brutal-danger btn-hapus" data-id="${item.id}">
            <i class="fas fa-trash"></i>
          </button>
        </div>
      </td>
    </tr>`;
  });

  html += `</tbody></table></div>`;
  $('#tableContainer').html(html);
}

```

### Penjelasan:

Fungsi loadData() dipanggil setiap kali halaman dimuat atau setelah ada perubahan data. Dia akan minta data ke server via AJAX, lalu renderTable() akan mengubah data JSON jadi HTML tabel. Setiap baris punya tombol Edit dan Hapus dengan atribut data-id yang berisi ID mahasiswa. Ini penting buat operasi edit dan hapus nantinya.

### e. AJAX (Edit Data)

#### Source Code:

```

$(document).on('click', '.btn-edit', function() {
  const id = $(this).data('id');

```

```

$.ajax({
  url: baseUrl + '/data',
  method: 'GET',
  success: function(response) {
    const data = response.find(item => item.id === id);
    if (data) {
      $('#id').val(data.id);
      $('#nama').val(data.nama);
      $('#nim').val(data.nim);
      $('#email').val(data.email);
      $('#jenis_kelamin').val(data.jenis_kelamin);
      $('#pendidikan').val(data.pendidikan);
      setHobiCheckboxes(data.hobi);
      isEdit = true;
      $('#modalTitle').text('Edit Mahasiswa');
      $('#btnSubmit').text('Update');
      openModal();
    }
  }
});
});

```

### Penjelasan:

Waktu tombol Edit diklik, saya ambil ID dari atribut data-id. Kemudian AJAX minta semua data ke server, cari data yang ID-nya cocok, lalu isi form dengan data tersebut. Setelah itu ubah judul modal jadi "Edit Mahasiswa" dan tombol submit jadi "Update". Dengan cara ini, satu form bisa dipake buat tambah dan edit.

## f. AJAX (Submit Form) - Add / Edit

### Source Code:

```

$('#formMahasiswa').on('submit', function(e) {
  e.preventDefault();
  const id = $('#id').val();
  const url = isEdit ? baseUrl + '/' + id : baseUrl;
  const method = isEdit ? 'PUT' : 'POST';

  const hobiValues = [];
  $('#input[name="hobi[]"]:checked').each(function() {
    hobiValues.push($(this).val());
  });

  const formData = {
    nama: $('#nama').val(),
    nim: $('#nim').val(),
    email: $('#email').val(),
    jenis_kelamin: $('#jenis_kelamin').val(),
    hobi: hobiValues,
    pendidikan: $('#pendidikan').val()
  };

  $.ajax({
    url: url,
    method: method,
    data: formData,
    headers: {
      'X-CSRF-TOKEN': $('meta[name="csrf-token"]').attr('content')
    },
    success: function(response) {
      showToast(response.message, 'success');
      closeModal();
      loadData();
    },
    error: function(xhr) {
      if (xhr.status === 422) {
        const errors = xhr.responseJSON.errors;

```

```

        let msg = '';
        $.each(errors, function(key, value) {
            if (key === 'hobi') {
                msg += 'Hobi: ' + value[0] + '\n';
            } else if (key === 'hobi.*') {
                msg += 'Hobi tidak valid\n';
            } else {
                msg += value[0] + '\n';
            }
        });
        showToast(msg.trim(), 'error');
    } else {
        showToast(xhr.responseJSON?.message || 'Terjadi kesalahan', 'error');
    }
}
});
});

```

### Penjelasan:

Ini bagian paling krusial. Waktu form disubmit, saya cek dulu apakah ini operasi tambah atau edit (dari variabel isEdit). Kalau edit, URL-nya pakai ID dan method-nya PUT. Kalau tambah, URL-nya base dan method-nya POST. Saya kumpulin semua data dari form, termasuk hobi yang dicentang (disimpan sebagai array). Setelah itu kirim via AJAX, kalau sukses muncul notifikasi dan tabel di-refresh. Kalau error validasi, tampilkan pesan errornya.

## 4. OUTPUT

### A. Main Page (Table Management)

| DATA   MAHASISWA<br>MANAJEMEN CRUD - JS ON LOCAL |                      |           |                    |        |                 |      |       |
|--------------------------------------------------|----------------------|-----------|--------------------|--------|-----------------|------|-------|
| DAFTAR MAHASISWA                                 |                      |           |                    |        |                 |      |       |
| ID                                               | NAMA                 | NIM       | EMAIL              | JK     | HOBI            | PEND | Aksi  |
| 1                                                | Muhammad Raiful Hana | 231102227 | raiful@example.com | PEJA   | Reading, Gaming | S1   | 👁️ 🗑️ |
| 2                                                | Ahmad Fauzi          | 231102228 | ahmad2@example.com | PEJA   | Reading, Gaming | S1   | 👁️ 🗑️ |
| 3                                                | Siti Aminah          | 231102229 | siti3@example.com  | WANITA | Reading, Gaming | S1   | 👁️ 🗑️ |
| 4                                                | Budi Santoso         | 231102230 | budi4@example.com  | PEJA   | Reading, Gaming | S1   | 👁️ 🗑️ |
| 5                                                | Dewi Lestari         | 231102231 | dewi5@example.com  | WANITA | Reading, Gaming | S1   | 👁️ 🗑️ |
| 6                                                | Rizky Ramadhan       | 231102232 | rizky6@example.com | PEJA   | Reading, Gaming | S1   | 👁️ 🗑️ |
| 7                                                | Putri Utami          | 231102233 | putri7@example.com | WANITA | Reading, Gaming | S1   | 👁️ 🗑️ |
| 8                                                | Fajar Nugroho        | 231102234 | fajar8@example.com | PEJA   | Reading, Gaming | S1   | 👁️ 🗑️ |
| 9                                                | Diana Putri          | 231102235 | diana9@example.com | WANITA | Reading, Gaming | S1   | 👁️ 🗑️ |

### B. Form Add



**TAMBAH MAHASISWA** X

**NAMA LENGKAP**  
Muhamad Santoso Agung

**NIM (10 D1617)**  
2311103485

**EMAIL**  
santoso.agung@example.com

**JENIS KELAMIN**  
Pria

**HOBBI (PILIH MINIMAL 1)**  
☐ Coding ☒ Gaming ☒ Membaca ☐ Menulis  
☐ Musik ☐ Traveling ☒ Olahraga ☐ Fotografi  
☒ Memasak

**PENDIDIKAN**  
S2

**BATAL** **SIMPAN**

### C. Form Edit

**EDIT MAHASISWA** X

**NAMA LENGKAP**  
Muhamad Rafiful Hana

**NIM (10 D1617)**  
2311102227

**EMAIL**  
raffifulhan@gmail.com

**JENIS KELAMIN**  
Pria

**HOBBI (PILIH MINIMAL 1)**  
☒ Coding ☐ Gaming ☒ Membaca ☐ Menulis  
☒ Musik ☒ Traveling ☐ Olahraga ☒ Fotografi  
☐ Memasak

**PENDIDIKAN**  
S1

**BATAL** **UPDATE**

### D. Delete

# DATA MAHASISWA

MANAJEMEN CRUD - JSON LOCAL

## DAFTAR MAHASISWA

+ TAMBAH DATA

| NO | NAMA                 | NIM        | EMAIL              | STATUS | KELOMPOK  | PEND      | Aksi |
|----|----------------------|------------|--------------------|--------|-----------|-----------|------|
| 1  | Muhammad Rafful Hana | 2311102227 | refi               | PRIA   | Traveling | Keagregat | S1   |
| 2  | Ahmad Fauzi          | 2311102228 | ahmad2@example.com |        | Membara   | Memara    | S1   |
| 3  | Siti Aminah          | 2311102229 | siti3@example.com  | WANITA | Musik     | Traveling | S1   |
| 4  | Budi Santoso         | 2311102230 | budi4@example.com  | PRIA   | Chakra    | Keagregat | S1   |
| 5  | Dewi Lestari         | 2311102231 | dewi5@example.com  | WANITA | Membara   | Gaming    | S1   |
| 6  | Rizky Ramadhan       | 2311102232 | rizky6@example.com | PRIA   | Coding    | Musik     | S1   |
| 7  | Putri Utami          | 2311102233 | putri7@example.com | WANITA | Traveling | Keagregat | S1   |
| 8  | Fajar Nugroho        | 2311102234 | fajar8@example.com | PRIA   | Membara   | Chakra    | S1   |
| 9  | Diana Putri          | 2311102235 | diana9@example.com | WANITA | Coding    | Gaming    | S1   |

Yakin ingin menghapus data mahasiswa ini?

Cancel

OK